Lab Manual for Digital Logic and Design

LAB-04 and 05

A) Verification of DE Morgan's Law
B) Development and Verification of XOR and XNOR gates using universalgates

EXPERIMENT NO. : 03

C) Verification of DE Morgan's Law

D) Development and Verification of XOR and XNOR gates using universalgates

Objective:

Boolean algebra uses many of the same laws as those of ordinary algebra. De Morgan's theorem allows for the simplification of a Boolean Expression by the cancellation of some redundant inversions. In this experiment, we'll know about these laws & theorem. Implement XOR and XNOR gates using NAND and NOR gates.

Apparatus:

- Logic trainer/MultiSim
- Connecting wires
- IC: 7400LS, 7408LS, 7404LS & 7432LS
- Power supply

Theory:

De Morgan developed a theorem that allows conversion between a logic expression that has inversions on the output to a different logic expression with the inversions on each of the inputs. This may allow for the simplification of a Boolean Expression by the cancellation of some redundant inversions. There are two Boolean Equations that represent De Morgan's Theorem:

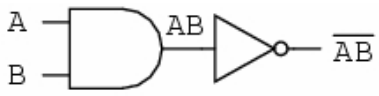
$$\overline{AB} = \overline{A} + \overline{B} \quad \text{.....(i)}$$

$$\overline{A+B} = \overline{A}\,\overline{B} \quad \text{.....(ii)}$$

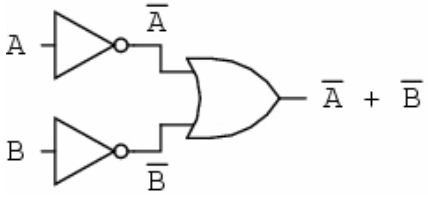
Procedure:

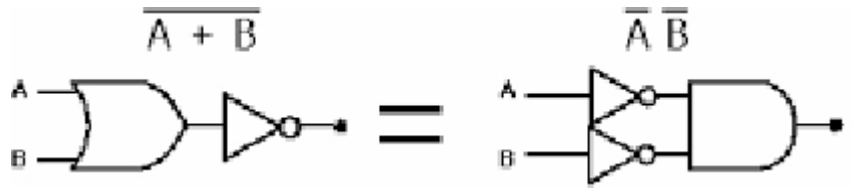
- At first construct the circuits shown in the Boolean laws.
- Check if the laws are valid. Give truth tables for each law.
- Apply various combinations of inputs as shown in the truth table and observe theconditions of LEDs.

Logic Diagram:



. . . is equivalent to . . .





Simulation:

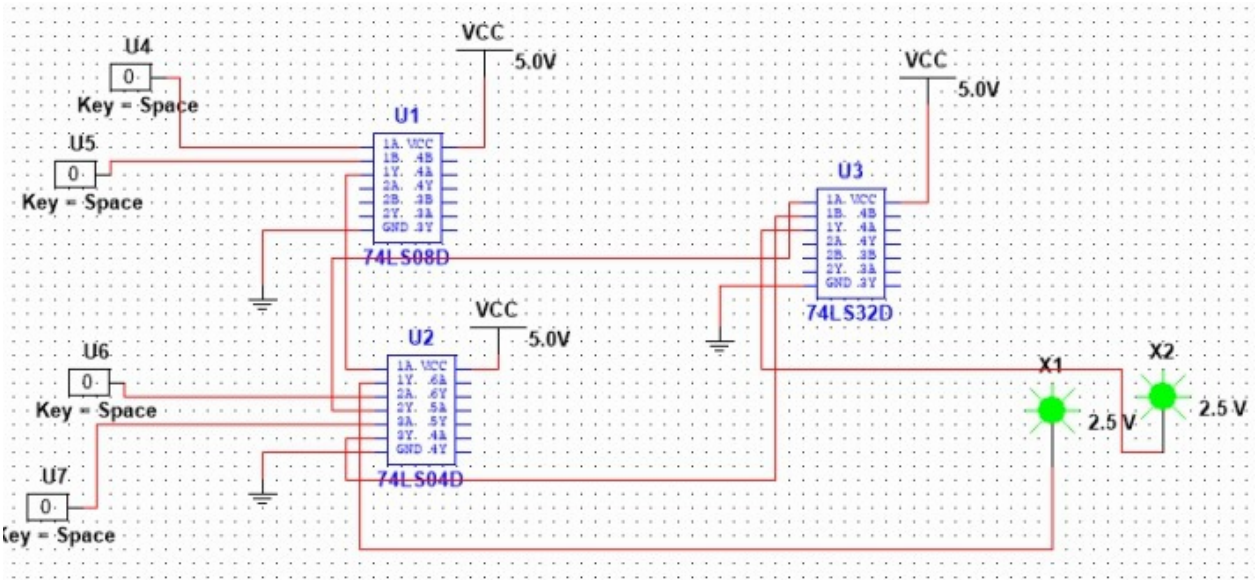


Fig:3.1.1 Verification of DE Morgan's Law

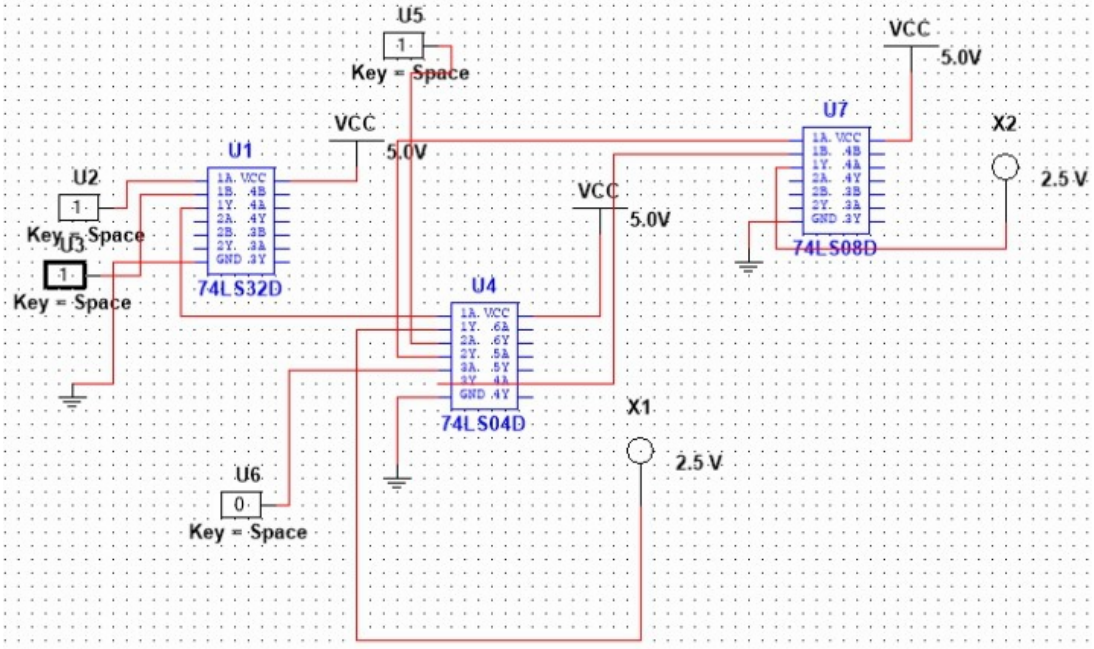


Fig:3.2.1 Verification of DE Morgan's Law

Observations & Calculations:

Truth table that verifies 1st Demorgan's law: (AB)'=A' + B'

A	*B*	*(AB)'*	*A' + B'*
0	*0*	*1*	*1*
0	*1*	*1*	*1*
1	*0*	*1*	*1*
1	*1*	*0*	*0*

Truth table that verifies 2nd Demorgan's law: (A+B)'=A'. B'

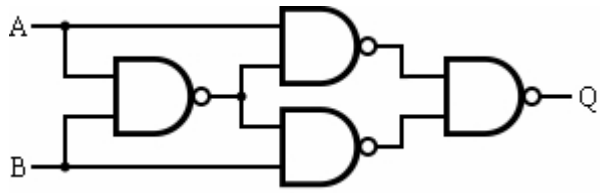
A	*B*	*(A+B)'*	*A'. B'*
0	*0*	*1*	*1*
0	*1*	*0*	*0*
1	*0*	*0*	*0*
1	*1*	*0*	*0*

NAND and NOR gates are called universal gates because all of the other gates may be constructed using only those two gates. That is important because it's a lot cheaper in practice to make lots of similar things than a bunch of different things (different gates).

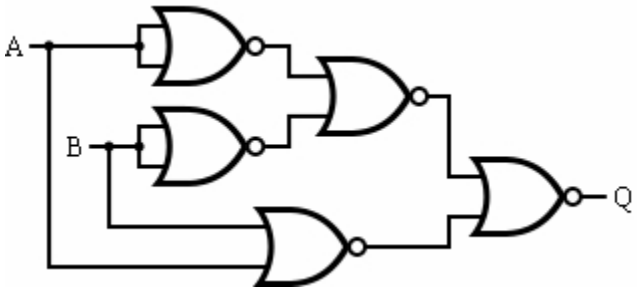
Implementation of XOR:

The XOR gate (sometimes EOR gate) is a digital logic gate that implements an exclusive disjunction. The XOR gate with inputs A and B implements the logical expression $A \cdot \overline{B} + \overline{A} \cdot B$

NAND implementation of XOR gate



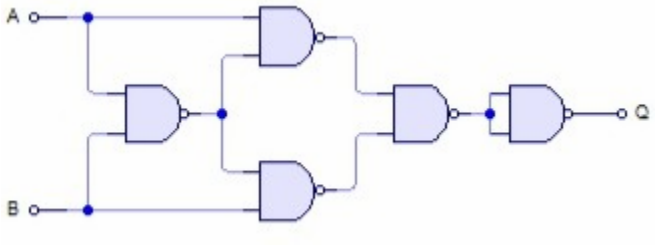
NOR implementation of XOR gate



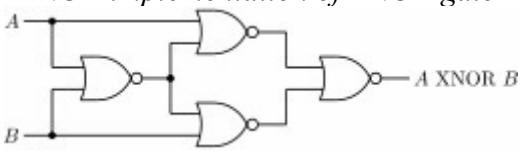
Implementation of XNOR:

The XNOR gate is a digital logic gate whose function is the inverse of the exclusive OR (XOR) gate. The XNOR gate with inputs *A* and *B* implements the logical expression $A \cdot B + \overline{A} \cdot \overline{B}$

NAND implementation of XNOR gate



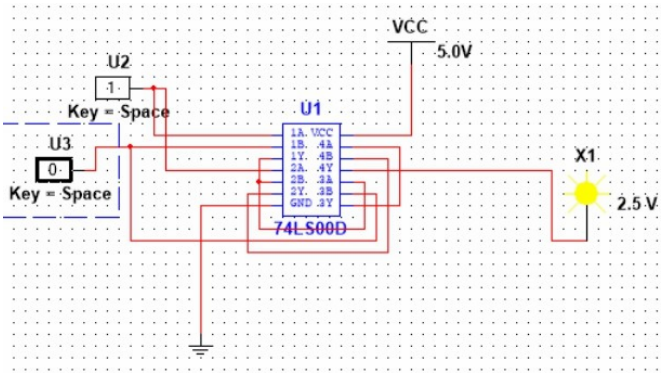
NOR implementation of XNOR gate



Procedure:

- Construct the circuits shown in logic diagrams.
- Connect inputs to switches of logic trainer and output to LED.
- Apply various combinations of inputs and observe the conditions of LEDs.
- Mark results in truth table.

Simulation:



Observations & Calculations:

NAND and NOR implementation of XOR gate truth table

Input A	***Input B***	***Actual output***	***NAND implemented circuit output***	***NOR implemented circuit output***
0	*0*	*0*	*0*	*0*
0	*1*	*1*	*1*	*1*
1	*0*	*1*	*1*	*1*
1	*1*	*0*	*0*	*0*

NAND and NOR implementation of XNOR gate truth table

Input A	***Input B***	***Actual output***	***NAND implemented circuit output***	***NOR implemented circuit output***

0	*0*	*1*	*1*	*1*
0	*1*	*0*	*0*	*0*
1	*0*	*0*	*0*	*0*
1	*1*	*1*	*1*	*1*

Conclusion:

Implement the following circuits in MultiSim. The outputs were observed on LED's and outputswere same as truth table.